# BONUS D · Dashboard de mercado en Flask (sin JavaScript)

**No es obligatorio para terminar el taller de 150 minutos.** Empezad solo después del flujo principal y de abrir PRs. Es una ampliación para equipos rápidos o para completar después.

## Caso de uso de negocio

Una empresa de compraventa quiere saber **cuál es el precio mediano anunciado** de los vehículos por estado (fair/good/excellent/like new) y **cuántos anuncios** hay en cada grupo. Esto permite contextualizar una tasación: ¿la estimación está dentro del rango observado? ¿En qué categorías existe menos muestra? No es una tasación profesional ni prueba causal de que el estado explique las diferencias.

## Requisito indispensable

Los datos tienen que salir **de la tabla PostgreSQL** `anuncios` que habéis creado con Jupyter, **no del CSV**. Si no se ha ejecutado el notebook de carga, esta vista debe advertirlo o informar del error, sin presentar datos inventados.

## Tareas a implementar (vosotros escribís el código)

### D1 · `app/analytics.py` — consulta y agregación

- Utilizar el `engine` SQLAlchemy proporcionado y consultar las columnas `condition`, `price` desde `anuncios`.
- Calcular por cada uno de los cuatro estados la **cantidad de anuncios** y la **mediana de precios**. La mediana es útil porque existen precios extremos.

- Devolver un `pandas.DataFrame` con los nombres exactos `condition`, `cantidad`, `precio_mediano`, ordenado por `ORDEN_ESTADOS`.
- Descartar valores vacíos y comprobar que no se mezclan categorías fuera del conjunto definido.

### D2 · `app/analytics.py` — generación de imagen

- Crear una figura con Matplotlib (`matplotlib` está preconfigurado con backend `Agg`). Dibujar al menos **una gráfica de barras**: mediana de precio USD por estado.
- Ampliación opcional: segundo gráfico con el número de anuncios por estado (no confundir con número de tasaciones del usuario).
- Añadir título, nombres de ejes y etiquetas que permitan interpretar el gráfico. La escala de precio debe comenzar en cero para las barras.
- Guardar la figura en un `BytesIO` como PNG, cerrar los recursos gráficos y devolver los bytes. `plt.show()` no sirve para un endpoint.

### D3 · `app/main.py` — rutas HTTP

- Implementar `GET /estadisticas` para renderizar `templates/estadisticas.html`.
- Implementar `GET /estadisticas/grafico.png`: obtener los datos desde la función D1, llamar a D2 y devolver los bytes con tipo MIME `image/png`. En Flask es posible usar `Response` o `send_file`.
- El dashboard debe indicar de forma amable que se ejecute el notebook si aún no existe `anuncios`.

### D4 · `templates/estadisticas.html` — visualización

- Mostrar el título, la pregunta de negocio, al menos una imagen cargada desde `/estadisticas/grafico.png` y una nota sobre el origen (50.000 anuncios de Craigslist, precios de anuncio en USD).
- Crear un enlace de navegación hacia `/estadisticas` desde `templates/base.html`.
- **Sin JavaScript**: el navegador puede cargar un PNG generado por Flask mediante una etiqueta HTML de imagen.

## Prueba de aceptación (bonus)

1. Ejecutar primero Jupyter; comprobar en SQL `SELECT COUNT(*) FROM anuncios` = 50000.

2. Abrir `/estadisticas`: aparece título y una gráfica en lugar de una imagen rota.
3. Abrir `/estadisticas/grafico.png` por separado: devuelve HTTP 200 y `Content-Type: image/png`.
4. Comprobar que la gráfica cambia al consultar una base de datos con otros registros; si siempre muestra cifras fijas, no vale.
5. Indicar en `ENTREGA_GRUPO.md` la URL del PR del bonus y quién lo revisó.

## División del bonus en Git

Quien lo implemente crea una rama `feature/dashboard`, realiza commits y abre PR. Como el bonus requiere añadir rutas a `app/main.py`, empezadlo **después de fusionar** los PRs obligatorios. En equipos de 2 o 3, todo el mundo debe haber abierto ya un PR individual por el núcleo.